

VELoop Rewards — Daily Streak System

Full-Stack MERN Internship Project

Project Type: Full-Stack Web Development

Primary Focus: Backend-driven streak logic, reward security, authentication, API integration, responsive UI and design implementation

Mandatory Stack: MERN + Bootstrap + CSS Modules

Project Assigned On: Today, When it Assigned

Duration: 20 – 24 Days from its assigned date

Submission Time: 5:30 PM IST

Assets: https://drive.google.com/drive/folders/1R9K5Epc7MINqS-6ZUCse_3smxVU4I-QU?usp=sharing

1. Project Overview

Build a complete **Daily Streak & Rewards system** for VELoop Rewards.

The feature allows an authenticated user to log in every day, maintain a consecutive-day streak and unlock increasingly valuable rewards.

The provided designs show the intended VELoop Rewards Daily Streak experience for:

- **Desktop**
- **Mobile**

The final implementation must reproduce the provided design language very closely while making the entire feature **fully functional through a MERN backend**.

This is **not a frontend-only design task**.

The intern must build:

React Frontend



Express.js API



Node.js

↓
MongoDB

Every important value and business decision must come from the backend.

2. Core Principle

The frontend must never be the source of truth.

The browser can be manipulated.

Therefore, the following must **NOT** be controlled by React:

- Current streak
- Current day
- Today's claim status
- Previous-day claim status
- Reward values
- Reward type
- Reward currency
- 24-hour countdown
- Unlock time
- Missed-day detection
- Streak reset
- User identity
- Claim eligibility
- Reward transaction
- Reward balance
- Claim status

The backend must calculate and validate these values.

3. Example of What Must NOT Happen

Do not implement:

```
const streak = 3;  
const currentDay = 4;  
const reward = 20;
```

or:

```
setStreak(streak + 1);
```

or:

```
localStorage.setItem("streak", "7");
```

as the actual streak system.

A user must not be able to open DevTools and change:

Day 2 → Day 7

or:

Timer → 00:00

or:

Claimed → Unclaimed

and obtain a reward.

4. Required User Experience

The complete flow should be:

User Authentication



Daily Streak Page



Backend checks user's streak



Backend determines current day



Backend determines today's eligibility



Frontend displays correct state



User clicks Claim



Backend validates everything



CPA Ad Demo State



Claim confirmed

↓
Backend grants reward
↓
Backend records transaction
↓
Next day becomes locked
↓
24-hour backend-controlled timer
↓
Next day unlocks

5. Daily Streak Logic

The streak consists of **7 reward days**, matching the supplied design.

Example:

Day 1
Day 2
Day 3
Day 4
Day 5
Day 6
Day 7

The user progresses through the days consecutively.

6. Day 1

When an eligible new user opens the Daily Streak page:

Current Streak:

1 Day

Day 1:

Available / Claimable

The user clicks:

Claim Reward

Before granting anything, the backend validates:

Authenticated user?

Today's check-in allowed?

Current streak valid?
Day 1 eligible?
Already claimed today?

Only if everything passes should the reward process continue.

7. CPA Advertisement Demo

After the user clicks the claim button, show a **CPA Advertisement Demo State**.

The intern does **not** need to integrate a real CPA/ad network.

Instead, create a polished placeholder such as:

Preparing your reward...

Advertisement / Reward Verification

Please wait...

The visual experience should match VELoop's theme.

Important

The ad/demo state must **not itself grant the reward**.

The backend must ultimately control the claim.

Our team will integrate the real CPA/ad solution later.

8. Day 1 Successful Claim

After successful backend validation and the demo CPA state:

Day 1
✓ Claimed

The backend updates the streak state.

Example:

Current Streak:

1

and:

Day 1:
CLAIMED

The frontend then refreshes its state from the backend.

Do not simply change the React state and assume the operation succeeded.

9. Day 2 — 24-Hour Timer

After Day 1 is claimed, Day 2 becomes the next target.

However, it must remain locked until the appropriate time.

The design should show something like:

Day 2

Next reward

24:00:00

or an appropriate countdown presentation.

10. Timer Must Come From Backend

This is a **critical requirement**.

Do NOT simply do:

```
setInterval(() => {  
  timer--;  
}, 1000);
```

based on a hardcoded 24-hour value.

The backend should provide something like:

```
{  
  "nextClaimAt": "2026-09-06T18:30:00.000Z",
```

```
"serverTime": "2026-09-05T18:30:00.000Z"  
}
```

The frontend can visually count down using those server-provided timestamps.

11. Why Server Time Is Required

A user could otherwise manipulate their device clock:

Phone time:
11:59 PM

then change it to:

12:00 AM

and attempt to claim early.

The backend must use its own trusted server/database time to determine whether Day 2 is actually available.

Frontend countdown = visual representation.

Backend timestamp = actual authority.

12. Backend Claim Validation

Every time the user presses Claim, the backend must independently verify:

```
graph TD
    A[User authenticated?] --> B[Streak exists?]
    B --> C[Current day correct?]
    C --> D[Previous day claimed?]
    D --> E[Required waiting period completed?]
    E --> F[Today's reward not already claimed?]
    F --> G[User account eligible?]
    G --> H[ ]
```

Grant reward

The frontend's displayed countdown cannot be trusted.

13. Previous-Day Validation

Before every claim, the backend must check the previous day's status.

Example:

Day 1 → Claimed
Day 2 → Claimable

Day 2 can proceed.

But:

Day 1 → Not Claimed
Day 2 → User tries to claim

The backend must reject it.

14. Sequential Claiming

The user must not be able to jump from:

Day 1

directly to:

Day 5

by changing a request.

For example, sending:

```
{  
  "day": 5  
}
```

must not cause Day 5 to be granted.

The backend determines the user's actual current streak day.

15. Missed Day Logic

This is one of the most important business rules.

If the user fails to claim the next required streak reward within the required window, their streak must reset.

Example:

Day 1 → Claimed

Day 2 → Claimed

Day 3 → Missed

Then when the user returns later:

Streak:

RESET

Current Day:

Day 1

The backend must determine the reset.

The frontend must not decide:

if (missed) streak = 1;

16. Example Missed-Day Scenario

Day 1

✓ Claimed

Day 2

✓ Claimed

Day 3

User does not return

Required claim window passes

↓

Backend detects missed streak

↓

Streak resets

↓

Next visit:

Day 1

Claim Now

17. Streak Count

The UI must display the current streak count.

Example:

🔒 7 Day Streak

Keep it going!

or:

🔒 3 Day Streak

depending on the actual backend state.

The streak number must come from the backend.

18. Never Trust streak From Frontend

A malicious request like:

```
{  
  "streak": 7  
}
```

must be ignored.

The backend determines:

Actual streak = 2

based on stored claim history.

19. Reward Data

The rewards displayed in the supplied design must be represented in the system.

The design currently illustrates:

Day	Reward
Day 1	+5 VEs
Day 2	+10 VEs
Day 3	+15 VEs
Day 4	₹1 Amazon Gift Card
Day 5	₹2 Amazon Gift Card
Day 6	+30 VEs
Day 7	₹5 Amazon Gift Card

These values should **not be hardcoded into React**.

They should exist in backend configuration/database data.

20. Reward Configuration

Create a backend-driven structure such as:

```
StreakReward
├── day
├── rewardType
├── currency
├── amount
├── title
├── description
├── asset
├── active
└── metadata
```

Example:

Day: 2
Type: VES

Amount: 10

The frontend receives the reward configuration through the API.

21. Reward Assets

The backend can return an asset identifier/type:

coin
gift-card
crown

The actual static visual asset remains in the frontend.

This is acceptable because an image file itself is not a business value.

The **reward identity/value** must come from the backend.

22. Ultimate Reward

The design highlights Day 7 as:

Ultimate Reward
₹5
Amazon Gift Card

The backend must determine:

- Day 7 reward
- Reward type
- Reward amount
- Unlock day
- Claim status

The frontend simply renders the returned data.

23. Top Reward Section

The large reward area shown in the design should be populated from backend data.

It should dynamically show information such as:

Ultimate Reward
₹5
Amazon Gift Card
Unlock on Day 7

If the backend configuration changes, the UI should update without changing React source code.

24. Statistics

The supplied design includes statistics such as:

Total Rewards
7

Checked In
1

Next Reward
+10 VEs

These must be backend-driven.

For example:

```
{
  "totalRewards": 7,
  "checkedIn": 1,
  "nextReward": {
    "amount": 10,
    "currency": "VES"
  }
}
```

The exact response structure is up to the intern.

25. Total Rewards

"Total Rewards" should represent the configured streak reward cycle or the appropriate platform-defined value represented in the design.

Do not blindly hardcode:

7

inside React.

The backend should provide the value.

26. Checked-In Count

The backend should calculate how many streak days the user has successfully claimed/check-in completed according to the current cycle.

Example:

Day 1 → Claimed

Day 2 → Claimed

Day 3 → Not yet

Checked In:

2

27. Next Reward

The backend determines the user's next eligible reward.

Example:

Current:

Day 1

Next:

Day 2

Reward:

+10 VEs

The frontend displays the backend response.

28. Daily Reward Cards

The page must contain the 7 reward cards represented in the design.

Each card should dynamically show:

- Day number
 - Badge where applicable
 - Reward image
 - Reward title
 - Reward value
 - Reward currency/type
 - Claim state
 - Locked state
 - Today's state
 - Claimed state
-

29. Day States

At minimum, support:

LOCKED
AVAILABLE
TODAY
CLAIMED
MISSED

The backend should determine the state.

30. Day 1 Example

Before claiming:

Day 1

Daily Reward

+5
5 VEs

Claim Reward

After claiming:

Day 1

Daily Reward

+5

5 VEs

✓ Claimed

31. Current Day Highlight

The design highlights today's actionable day using a prominent visual treatment.

For example:

TODAY

The backend must determine which day is "today's actionable streak day."

Do not allow:

?day=7

to change the user's actual current day.

32. Locked Days

Future days should display something similar to:

🔒 Locked

The frontend should not decide which days are locked.

The backend should return the state.

33. Claimed Days

Previously completed days should display:

✓ Claimed

and must not be claimable again.

34. Claim Button

The claim button must be dynamically controlled.

If the backend says:

AVAILABLE

the frontend shows:

Claim Reward

If:

LOCKED

show:

Locked

If:

CLAIMED

show:

Claimed

35. Backend Claim Endpoint

A possible endpoint:

POST /api/daily-streak/claim

The request should be minimal.

For example:

```
{  
  "day": 2  
}
```

is acceptable only if the backend treats it as a requested action and independently verifies that Day 2 is actually the user's current eligible day.

Better yet, the backend can derive the day entirely from the user's streak state.

36. Never Trust Reward Data

This must fail:

```
{  
  "day": 2,  
  "reward": 100000,  
  "currency": "VES"  
}
```

The backend must ignore client-provided:

reward
currency
amount
streak

and retrieve the actual configured reward.

37. Reward Transaction

Every successful claim must create a reward transaction.

Example:

Streak Claim

+10 VEs

Source:
DAILY_STREAK

Day:
2

Reference:
STREAK-XXXXXXXX

The transaction should include:

transactionId
userId
rewardType
amount
currency
source
streakDay
referenceId
balanceBefore
balanceAfter
createdAt

38. Wallet Integration

The reward must be credited to the user's actual backend wallet.

For example:

Wallet
VEs: 100

Day 2 Claim
+10 VEs

Wallet
VEs: 110

The frontend must retrieve the updated balance from the backend.

39. No Fake Wallet Update

Do not implement:

```
setVEs(prev => prev + 10);
```

as the actual reward operation.

Correct:

Claim
↓
Backend
↓
Wallet update
↓
Transaction
↓
Backend success
↓
Frontend refreshes wallet

40. Idempotency

The claim API must be idempotent.

If the same request arrives twice:

POST /claim
POST /claim

the user must not receive:

+10
+10

Instead:

+10

only once.

41. Double-Click Protection

Frontend:

Claim
↓
Disable button

is good UX.

But it is **not security**.

Backend must independently prevent duplicate claims.

42. Concurrent Request Protection

Test:

Request A → Claim Day 2

Request B → Claim Day 2

sent simultaneously.

Only one should successfully grant the reward.

This should be handled with appropriate MongoDB atomic operations/transactions and unique constraints where appropriate.

43. Authentication

Authentication must be backend-driven.

Use:

JWT

or the project's approved authentication approach.

The Daily Streak APIs must be protected.

44. User Identification

Do not trust:

```
{  
  "userId": "USER123"
```

}

from the frontend for ownership.

The authenticated user's identity should be derived from the JWT/session.

45. Cross-User Attack

User A must not be able to submit:

User B
Day 7
Claim

and receive User B's reward.

Backend authorization must prevent this.

46. Server Time

All streak decisions must use server-side time.

The backend should maintain/return:

serverTime
nextClaimAt

where required.

This also allows the frontend countdown to remain synchronized.

47. Countdown Implementation

The frontend can calculate the visible remaining time using:

nextClaimAt - current server-relative time

The intern should avoid trusting the device's clock.

Example:

Backend:

nextClaimAt = 2026-09-06 10:00

Frontend:

Countdown → 14:32:17

If the user changes their system time, it should not unlock the claim.

48. Timer Completion

When the timer reaches zero visually, the frontend should request fresh streak state from the backend.

Do not immediately assume:

Timer = 0

→ claim allowed

Instead:

Timer ≈ 0

↓

GET /daily-streak/status

↓

Backend confirms availability

↓

UI changes to Claim Reward

49. Missed-Streak Detection

The backend should compare relevant timestamps/claim history.

For example:

Previous claim:

Monday 10:00

Required next claim window:

Tuesday

User returns:

Thursday

Backend determines that the streak was missed and resets it according to the platform rule.

50. Reset State

After a missed streak:

Current Streak:
0 / reset state

and the next eligible reward should be:

Day 1

The UI should update automatically from backend state.

51. Important Edge Case

A user must not be reset simply because the browser was closed.

The reset should happen based on **server-side streak rules**, not frontend activity.

52. Important Edge Case — Multiple Tabs

A user opens:

Tab A
Tab B

and claims in Tab A.

Tab B must not still be able to claim the same reward successfully.

The backend remains authoritative.

53. Important Edge Case — Refresh

After claiming Day 1:

Refresh browser

must show:

Day 1 → Claimed

Day 2 → Locked/Countdown

based on backend state.

Not:

Day 1 → Claim Now

because React state reset.

54. Important Edge Case — Logout/Login

After logging out and logging in again, the same streak state must be retrieved from MongoDB.

55. Daily Streak API

Recommended APIs:

GET /api/daily-streak

GET /api/daily-streak/status

POST /api/daily-streak/claim

GET /api/daily-streak/history

The intern can design a better REST structure.

56. Suggested GET Response

For example:

```
{
  "success": true,
  "serverTime": "2026-09-05T12:00:00Z",
  "streak": {
    "currentStreak": 1,
    "currentDay": 1,
    "checkedIn": 1,
    "totalRewards": 7,
    "nextClaimAt": "2026-09-06T12:00:00Z"
  },
  "rewards": []
}
```

The exact schema is up to the intern.

57. Backend Must Return Card States

The backend can return:

```
{
  "day": 1,
  "status": "CLAIMED"
}
{
  "day": 2,
  "status": "LOCKED",
  "nextClaimAt": "..."
}
{
  "day": 3,
  "status": "LOCKED"
}
```

This prevents frontend business logic from becoming authoritative.

58. Streak History

Create a backend history of check-ins.

Example:

StreakHistory

userId
day

cycleId
claimedAt
reward
transactionId
status

This allows the backend to determine:

- Previous claim
- Missed day
- Current streak
- Claimed rewards
- Cycle completion

59. Streak Cycle

A useful architecture is to treat each 7-day streak as a cycle.

Example:

Cycle 001

Day 1 ✓
Day 2 ✓
Day 3 ✓
Day 4 -
Day 5 -
Day 6 -
Day 7 -

After completion:

Cycle 001 → Completed

and a new cycle can begin according to the platform's rules.

The intern should document their chosen approach.

60. Database Models

At minimum:

User
Wallet
StreakConfig
StreakReward
StreakCycle
StreakClaim / StreakHistory
WalletTransaction
AuditLog

The intern may combine models where appropriate.

61. StreakReward Model

Example:

StreakReward

- day
- rewardType
- currency
- amount
- title
- subtitle
- assetType
- active
- metadata

62. StreakClaim Model

Example:

StreakClaim

- claimId
- userId
- cycleId
- day
- rewardId
- status
- claimedAt
- transactionId
- createdAt

A unique constraint such as:

userId + cycleId + day

should be considered to prevent duplicate claims.

63. WalletTransaction Model

Example:

WalletTransaction

```
|
├── transactionId
├── userId
├── currency
├── type
├── amount
├── source
├── referenceId
├── balanceBefore
├── balanceAfter
├── status
└── createdAt
```

64. Audit Log

Record important events:

```
STREAK_CLAIM_REQUEST
STREAK_CLAIM_SUCCESS
STREAK_CLAIM_REJECTED
STREAK_RESET
DUPLICATE_CLAIM
INVALID_CLAIM
```

This will help the development team investigate suspicious activity.

65. Backend Service Architecture

Do not put all logic inside routes.

Suggested:

```
services/
├── streak.service.js
```

```
|— reward.service.js
|— wallet.service.js
|— transaction.service.js
```

Example:

```
getStreakStatus()
calculateCurrentDay()
checkEligibility()
checkMissedDay()
claimReward()
resetStreak()
```

66. Configuration Must Be Backend Driven

The backend should control:

- Number of days
- Reward values
- Reward types
- Claim interval
- Streak rules
- Reset rules
- Active/inactive rewards

If the reward changes from:

Day 2 = 10 VEs

to:

Day 2 = 15 VEs

the frontend should not require a code change.

67. No Hardcoded 24 Hours in Business Logic

Avoid making:

```
const TWENTY_FOUR_HOURS = 86400000;
```

the final authority.

The backend should calculate the actual next eligible timestamp based on the stored claim time and configured streak rules.

68. CPA Demo State

Create a polished demo component:

Preparing your reward...

The backend may return a claim-processing state.

Our VELoop team will later replace this with the actual CPA integration.

Do not integrate:

- Real CPA network
 - Real ad SDK
 - Production ad credentials
 - Third-party ad tracking
-

69. Loader

When the user first opens the Daily Streak page, show a **VELoop-themed loader**.

For example:

VELoop Logo / Reward Icon

Loading your streak...

The loader should follow the provided design theme.

The original design direction also emphasizes polished loading/interaction states rather than a generic browser loading screen.

70. Skeleton Loading

After the initial loader or during API fetching, implement skeleton states for:

- Header
- Reward banner
- Statistics
- Ultimate reward
- Daily cards
- Information section

Do not show a blank page.

71. Skeleton Must Match the Theme

Avoid plain:

gray rectangle
gray rectangle
gray rectangle

The skeleton should fit the dark VELoop visual system.

72. Desktop Design

The supplied desktop design must be used as the primary visual reference.

It contains:

- Daily Streak header
- Streak badge
- Daily Check-In hero
- Reward information
- Statistics
- Ultimate Reward
- Seven reward cards
- Supporting benefits
- Footer/trust strip

The implementation should reproduce this composition closely.

73. Mobile Design

The supplied mobile design must be implemented as a dedicated responsive layout.

Do not simply shrink the desktop page.

The mobile version should preserve:

- Visual hierarchy
- Cards
- Rewards
- Streak information
- Buttons
- Navigation
- Footer/trust elements

while adapting dimensions and arrangement for small screens.

74. Tablet Design

There is no separate tablet reference image, so the intern must intelligently interpolate between the provided mobile and desktop designs.

Support:

768px
820px
834px
1024px

The layout should not simply become a stretched mobile page or compressed desktop page.

Use an appropriate tablet grid.

75. Responsive Breakpoints

The application must work from approximately:

320px+

through large desktop displays.

Test at:

320

360

375

390

414

480

768

820

834

1024

1280

1366

1440

1920+

76. Design Fidelity

The final UI should be **very close to the provided designs**.

Pay particular attention to:

- Dark background
- Purple/violet visual language
- Gold/yellow reward highlights
- Green success states
- Card borders
- Glow levels
- Typography
- Spacing
- Icon sizes
- Reward illustrations
- Button shapes
- Streak indicators
- Day cards

- Hero composition

Do not replace the design with a generic Bootstrap dashboard.

77. Color Theme

Use the supplied design's color theme as the source of truth.

The visual language should remain:

Dark + Purple + Gold + Green success accents

Avoid:

- Neon overload
 - Random gradients
 - Excessive glow
 - Crypto-style design
 - Casino styling
 - Children's-game styling
-

78. Assets

All assets supplied with the design should be used where applicable.

Do not replace the provided assets with random stock images.

The supplied visuals include reward-oriented artwork such as:

- Coins
 - Gift box
 - Crown
 - Calendar
 - Gift card
 - Icons
 - VELoop branding
-

79. Static Image Assets → Animation

Some assets are supplied as static images, particularly assets such as:

- Day 4 gift
- Day 7 crown
- Top-right/hero reward imagery

The intern may use an AI/design tool such as **Google Flow or another suitable image/video generation or animation tool** to create a subtle animated version of these assets.

For example:

Static Crown PNG



AI animation



Subtle floating/glow/rotation



Web-compatible animation

or:

Static Gift



Subtle shine / floating motion

80. Animation Rules

Animations must remain premium.

Good:

Subtle floating

Soft shine

Small rotation

Gentle glow

Reward reveal

Avoid:

Constant shaking

Extreme flashing

Huge particle explosions

Casino-style effects

Distracting movement

The goal is to enhance the supplied design, not change its visual identity.

81. AI Asset Usage

If an asset needs animation:

1. Use the supplied design as the reference.
2. Animate only the relevant individual asset.
3. Preserve its overall appearance.
4. Export an appropriate web format.
5. Optimize it.
6. Integrate it into the React component.

Do **not** upload the entire webpage screenshot to an AI tool and use the generated result as the webpage itself.

82. Navbar

Create the navbar/header shown in the provided design.

It must include the relevant:

- Back/navigation action
- Daily Streak title
- Streak indicator
- Gem/reward balance where shown
- Responsive mobile behavior

The displayed values should come from backend data wherever they represent user/account state.

83. Reward Balance

If the header displays the user's Gem balance:

the value must be fetched from the backend.

Do not use:

```
const gems = 120;
```

84. Navigation

The back button should function properly.

Example:

Daily Streak
←

The intern should use React Router or the application's routing system.

85. Icons

Use:

- **Lucide React**
- **React Icons**

where suitable.

Do not download random icon screenshots.

Icons should be consistent in:

- Stroke
 - Size
 - Weight
 - Alignment
-

86. CSS Requirement

Use:

Mandatory

- CSS Modules
- Bootstrap

The intern may use additional CSS where required.

Example:

DailyStreak.module.css

Avoid placing the entire page styling into one massive global CSS file.

87. Component Architecture

A possible structure:

```
DailyStreak/  
├── DailyStreakPage.jsx  
├── DailyStreak.module.css  
├── StreakHeader.jsx  
├── HeroBanner.jsx  
├── StreakStats.jsx  
├── UltimateReward.jsx  
├── RewardGrid.jsx  
├── RewardCard.jsx  
├── ClaimModal.jsx  
├── CpaDemo.jsx  
├── StreakLoader.jsx  
├── StreakSkeleton.jsx  
├── WhyStreak.jsx  
└── TrustFooter.jsx
```

This is only a recommendation.

A better architecture is welcome.

88. Frontend API Layer

Create a centralized API layer.

For example:

```
services/  
└─ streakApi.js
```

Functions could include:

```
getStreak()  
claimStreak()  
getHistory()  
getWallet()
```

Do not put API requests randomly throughout components.

89. Complete Data Flow

The preferred architecture is:

```
MongoDB  
↓  
Node/Express  
↓  
REST API  
↓  
React API Service  
↓  
React State  
↓  
UI
```

Not:

```
React  
↓  
Hardcoded Data
```

90. Loading Flow

When opening the page:

```
Open /daily-streak  
↓  
Veloop branded loader
```

↓
GET /api/daily-streak
↓
Backend calculates state
↓
Skeleton/content
↓
Final page

91. Claim Flow

User clicks Claim
↓
Frontend disables button
↓
POST /claim
↓
Backend validates
↓
CPA Demo
↓
Backend confirms reward
↓
Wallet transaction
↓
Streak updated
↓
Frontend requests latest state
↓
Day becomes Claimed
↓
Next day countdown appears

92. Claim Errors

Handle errors gracefully.

Examples:

Already claimed

This reward has already been claimed.

Still locked

Your next reward is not available yet.

Previous day missed

Your streak has been reset. Start again from Day 1.

Authentication

Please log in to continue.

Server error

Unable to process your reward.
Please try again.

Do not expose:

MongoServerError
AxiosError
CastError

to the user.

93. Refresh After Claim

After a successful claim:

POST /claim
↓
Success
↓
GET /daily-streak
↓
Update entire UI

Do not manually update every card with frontend assumptions.

94. Full Backend State Example

A backend response could conceptually look like:

```
{  
  "success": true,  
  "serverTime": "2026-09-05T12:00:00Z",  
}
```

```

"streak": {
  "currentStreak": 1,
  "currentDay": 1,
  "checkedIn": 1,
  "totalRewards": 7,
  "status": "ACTIVE",
  "nextClaimAt": "2026-09-06T12:00:00Z"
},

"rewards": [
  {
    "day": 1,
    "status": "CLAIMED",
    "reward": {
      "type": "VES",
      "amount": 5
    }
  },
  {
    "day": 2,
    "status": "LOCKED",
    "nextClaimAt": "2026-09-06T12:00:00Z",
    "reward": {
      "type": "VES",
      "amount": 10
    }
  }
]
}

```

This is only an example; the intern should design the final API appropriately.

95. Backend Must Calculate Everything

Backend determines:

- Current streak
- Current day
- Claim status
- Locked status
- Reward
- Reward amount
- Reward currency
- Next claim time
- Missed status
- Reset
- Eligibility
- Wallet update
- Transaction

Frontend determines only:

How to display it

How to animate it

How to collect user interaction

96. Anti-Cheat Test

The evaluator should attempt:

Change streak in React DevTools

Expected:

No effect.

97. Timer Cheat Test

Change system clock forward by:

24 hours

Expected:

Cannot claim unless backend says the reward is available.

98. Reward Cheat Test

Send:

```
{  
  "day": 2,  
  "reward": 1000000  
}
```

Expected:

Backend ignores fake reward.

99. Day Cheat Test

Send:

```
{  
  "day": 7  
}
```

while only eligible for Day 2.

Expected:

Rejected.

100. User ID Cheat Test

Send:

```
{  
  "userId": "anotherUser"  
}
```

Expected:

Backend ignores/rejects unauthorized ownership manipulation.

101. Duplicate Claim Test

Send the same claim request twice.

Expected:

First:
SUCCESS

Second:
REJECTED / IDEMPOTENT RESPONSE

No duplicate reward.

102. Concurrent Claim Test

Send two simultaneous requests.

Expected:

+10 VEs

not:

+20 VEs

103. Missed Streak Test

Create a development test scenario:

Day 1 → Claimed
Day 2 → Claimed
Advance server/test time
Day 3 → Missed

Expected:

Streak reset
Current Day = Day 1

Document how this was tested.

104. Database Integrity

After every successful claim, there should be a traceable relationship:

Streak Claim
↓
Reward
↓
Wallet Transaction
↓
Wallet Balance

The evaluator should be able to inspect MongoDB and understand what happened.

105. No Production Access

The intern should build this using their own development environment.

Do not provide:

- Production MongoDB
- Production server credentials
- Production JWT secret
- Production API keys
- Production admin credentials
- Production payout credentials

Use:

Intern's MongoDB Atlas
Intern's deployment
Intern's environment variables

106. Deployment

Recommended:

Frontend

Vercel / Netlify

Backend

Render / Railway / equivalent

Database

MongoDB Atlas

The live frontend must communicate with the live backend.

107. Environment Variables

Backend:

PORT=
MONGO_URI=
JWT_SECRET=

Frontend:

VITE_API_URL=

Use:

.env.example

Never commit real secrets.

108. README

The repository must include a detailed:

README.md

Explain:

- Project overview
 - MERN architecture
 - Installation
 - Environment setup
 - Database schema
 - Streak logic
 - Claim logic
 - Timer architecture
 - Reset logic
 - Reward system
 - Wallet integration
 - Security
 - API endpoints
 - Testing
 - Deployment
 - Known limitations
-

109. API Documentation

Create:

API_DOCUMENTATION.md

Document:

GET /api/daily-streak
GET /api/daily-streak/status
POST /api/daily-streak/claim
GET /api/daily-streak/history

and any additional APIs created.

110. Postman Collection

Include a working Postman collection covering:

Login
Get streak
Get status
Claim Day 1
Get updated streak
Claim Day 2
Get history

and negative tests.

111. Testing Documentation

Create:

TESTING.md

Include:

- Normal claim
- Already claimed
- Locked reward
- Timer

- Missed day
 - Reset
 - Duplicate request
 - Concurrent request
 - Fake reward
 - Fake day
 - Fake user
 - Changed device time
 - Unauthorized request
-

112. UI Acceptance Criteria

The UI must:

- Closely match the supplied desktop design.
 - Closely match the supplied mobile design.
 - Have a proper tablet layout.
 - Use the same overall color theme.
 - Use the supplied assets.
 - Use polished icons.
 - Include responsive interactions.
 - Include loading states.
 - Include skeleton states.
 - Include error states.
 - Include claim states.
 - Include locked states.
 - Include countdown.
 - Include streak reset state.
 - Include CPA demo state.
-

113. Backend Acceptance Criteria

The backend must:

- Use MongoDB.
- Use Express.
- Use Node.js.
- Authenticate users.

- Store streak data.
 - Store reward configuration.
 - Store claim history.
 - Store wallet transactions.
 - Calculate current streak.
 - Determine current day.
 - Determine claim eligibility.
 - Determine next claim time.
 - Detect missed streak.
 - Reset streak.
 - Validate previous-day completion.
 - Prevent duplicate claims.
 - Prevent reward manipulation.
 - Prevent day manipulation.
 - Prevent user manipulation.
 - Use server-side time.
 - Handle concurrent requests safely.
-

114. Technology Restrictions

Mandatory

MongoDB
Express.js
React.js
Node.js
Bootstrap
CSS Modules

Allowed

Mongoose
JWT
React Router
React Icons
Lucide React
Framer Motion
Axios
Redux Toolkit / Zustand if genuinely required

Do not introduce an unrelated backend technology instead of MERN.

115. Design Asset Animation Requirement

For static assets that benefit from motion:

Day 4 Gift
Day 7 Crown
Hero Reward Artwork
Top-right Reward Visual

the intern may create a subtle animated version using:

- Google Flow
- Another appropriate AI animation tool
- After Effects
- Other legitimate design tools

The resulting animation must preserve the supplied design's visual identity.

116. No Entire-Page AI Replacement

Do not generate a completely different AI-designed page and submit it.

AI may be used for:

- Individual asset animation
- Asset cleanup
- Transparent background extraction
- Supporting visual preparation

The actual UI must be implemented by the intern in React.

117. Suggested Repository

```
veloop-daily-streak/  
|  
├── frontend/  
|   ├── src/  
|   └── components/
```

```
| | | | | pages/
| | | | |   | DailyStreak/
| | | | |   | services/
| | | | |   | hooks/
| | | | |   | context/
| | | | |   | assets/
| | | | |   | utils/
| | | | |   | package.json
| | | | |
| | | | | backend/
| | | | |   | src/
| | | | |   |   | config/
| | | | |   |   | controllers/
| | | | |   |   | middleware/
| | | | |   |   | models/
| | | | |   |   | routes/
| | | | |   |   | services/
| | | | |   |   | validators/
| | | | |   |   | utils/
| | | | |   | seed/
| | | | |   | package.json
| | | | |
| | | | | docs/
| | | | |   | API_DOCUMENTATION.md
| | | | |   | DATABASE.md
| | | | |   | SECURITY.md
| | | | |   | TESTING.md
| | | | |
| | | | | postman/
| | | | |
| | | | | README.md
| | | | | .gitignore
```

119. Critical Failure Conditions

The project may be rejected if:

- Streak is hardcoded.
- Rewards are hardcoded only in React.
- Timer is controlled entirely by browser time.
- User can change current day from frontend.
- User can change reward amount.
- User can claim a locked day.
- User can claim the same day twice.
- User can jump directly to Day 7.
- Missed streak does not reset.
- Previous-day validation is missing.
- Wallet update exists only in React.

- No MongoDB persistence.
- APIs are fake.
- No authentication.
- User can access another user's streak.
- UI is only a screenshot.
- Production credentials are committed.

120. Final User Journey

The finished application should demonstrate:

LOGIN



DAILY STREAK



Backend loads user state



Branded Loader



Skeleton



Daily Streak UI



Current streak displayed



Today's reward identified



User clicks Claim



Backend validates eligibility



CPA Advertisement Demo



Backend grants reward



Wallet transaction created



Day marked Claimed



Next day becomes locked



Backend supplies nextClaimAt



24-hour countdown



Timer reaches zero



Frontend asks backend for fresh status



Next day becomes available

↓
User claims again
↓
...
↓
If a required day is missed
↓
Backend resets streak
↓
Day 1 becomes available again

121. Final Quality Standard

Before submitting, ask:

Security

Can I change my streak using DevTools?

NO

Reward

Can I send a fake reward amount?

NO

Timer

Can I change my phone/PC clock and claim early?

NO

Day

Can I directly request Day 7?

NO

Duplicate

Can I claim the same day twice?

NO

Reset

Does a genuinely missed streak reset correctly?

YES

Database

Can I see the claim and reward transaction in MongoDB?

YES

UI

Does the page look extremely close to the supplied design?

YES

Responsive

Does it look polished on mobile, tablet and desktop?

YES

122. Final Submission Checklist

MERN Backend

- ☐ Node.js
- ☐ Express.js
- ☐ MongoDB
- ☐ Mongoose
- ☐ Authentication
- ☐ Streak model
- ☐ Reward configuration
- ☐ Claim history
- ☐ Wallet integration
- ☐ Transaction ledger
- ☐ Server-side timer
- ☐ Missed-day detection

- ☐ Streak reset
- ☐ Previous-day validation
- ☐ Duplicate protection
- ☐ Concurrent-request protection
- ☐ Rate limiting
- ☐ Validation
- ☐ Audit logs

React Frontend

- ☐ Daily Streak page
- ☐ Navbar/header
- ☐ Hero section
- ☐ Statistics
- ☐ Ultimate Reward
- ☐ Seven reward cards
- ☐ Current streak
- ☐ Claim button
- ☐ Claimed state
- ☐ Locked state
- ☐ 24-hour countdown
- ☐ CPA demo state
- ☐ Loading screen
- ☐ Skeleton loading
- ☐ Error state
- ☐ Reset state
- ☐ Mobile design
- ☐ Tablet design
- ☐ Desktop design
- ☐ Bootstrap
- ☐ CSS Modules
- ☐ Lucide/React Icons
- ☐ Supplied assets
- ☐ Animated assets where appropriate

Testing

- ☐ Correct claim
- ☐ Duplicate claim
- ☐ Concurrent claim
- ☐ Locked day
- ☐ Fake day
- ☐ Fake reward
- ☐ Fake streak
- ☐ Fake user
- ☐ Timer manipulation
- ☐ Missed day
- ☐ Streak reset
- ☐ Refresh state
- ☐ Multiple tabs

Submission

- ☐ GitHub repository
 - ☐ Live frontend
 - ☐ Live backend
 - ☐ MongoDB development database
 - ☐ README
 - ☐ API documentation
 - ☐ Database documentation
 - ☐ Security documentation
 - ☐ Testing documentation
 - ☐ Postman collection
 - ☐ .env.example
 - ☐ Demo credentials
 - ☐ Demo video
-

123. Project Timeline & Submission Deadline

Project Assigned On: 5 September 2026

Project Start Date: 5 September 2026

Submission Deadline: 10 September 2026

Submission Time: 5:00 PM IST

124. DESIGN REFERENCE — ATTACHED AT THE END

DESKTOP DESIGN

The **desktop image attached at the end of this document** is the primary desktop visual reference.

The intern must reproduce the same:

- Overall layout
- Dark background
- Purple/gold visual theme
- Header
- Streak indicator
- Hero banner
- Statistics cards
- Ultimate reward section
- Seven-day reward grid
- Supporting information section
- Trust/footer section
- Typography hierarchy
- Card treatment
- Borders
- Shadows
- Icons
- Illustrations

- Button styling

Do not treat the image as a screenshot to place into the application.

Recreate it as real React components.

125. MOBILE DESIGN

The **mobile image attached at the end of this document** is the mobile visual reference.

The intern must create a responsive mobile implementation matching the supplied design rather than merely shrinking the desktop layout.

Pay special attention to:

- Header spacing
 - Reward balance
 - Hero proportions
 - Card widths
 - Day-card arrangement
 - Touch-friendly buttons
 - Typography
 - Vertical spacing
 - Bottom sections
 - Horizontal overflow
-

126. TABLET DESIGN

No separate tablet reference is provided.

Therefore, the intern must create a **responsive tablet adaptation between the supplied mobile and desktop designs**.

The result should feel intentionally designed for tablets.

127. FINAL OBJECTIVE

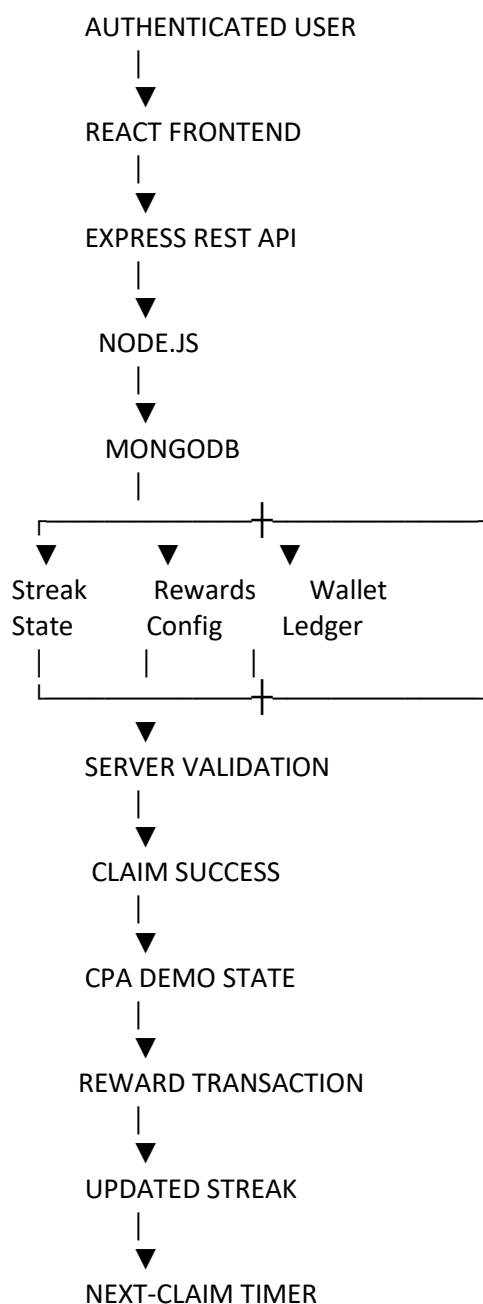
This project is not simply:

"Make a Daily Streak UI."

It is:

Build a complete, secure, backend-driven Daily Streak and Rewards system for VELoop Rewards using the MERN stack, while reproducing the supplied VELoop design with production-quality responsive UI.

The final architecture should follow:



The fundamental rule:

React controls the presentation. The backend controls the streak.

And the final experience should feel like a genuine VElow Rewards feature:

premium, rewarding, trustworthy, responsive, polished and fully functional
— not a static frontend mockup.





Daily Streak 🔥



120



Login Daily & Earn Bigger Rewards!

Maintain your streak and unlock exciting rewards every day.



1 Day Streak



Streak Calendar >



Total Rewards

7



Checked In

1



Next Reward

+10 VEs



Ultimate Reward

₹5



Amazon Gift Card



Unlock on
Day 7

✦ Come back tomorrow for more rewards! ✦

Day 1



Daily Reward

+5

5 VEs

Claimed

Day 2

Today



Daily Reward

+10

10 VEs

Claim Now >

Day 3



Daily Reward

+15

15 VEs

Locked

Day 4



Daily Reward

₹1

Amazon Gift Card

Locked

Day 5

Gift Card



Daily Reward

₹2

Amazon Gift Card

Locked

Day 6

Coin



Daily Reward

+30

30 VEs

Locked

Day 7

VIP



Ultimate Reward

₹5

Amazon Gift Card

Locked

Why Maintain Your Streak? ✦



Stay Active

Keep your streak alive & earn more!



Bigger Streak

More consecutive logins, bigger rewards!



Exclusive Rewards

Get coins, gift cards & special bonuses!



Don't Miss Out

Come back every day & unlock all rewards!



Official rewards only on VeloopRewards.in
Stay active, stay rewarded!



